# When the delta is fake

Three public agent-eval tasks with nop starters, frozen controls, and a scored fail trace

**Gyanu Mayank** · Independent · 31 October 2026 · v0.1.0
Code: github.com/gyanu2507/agent-eval-suite
Companion PRs: Harbor #4003, Terminal-Bench #2480 (`lease-fencing-reaper`)

---

**ABSTRACT**

Agent benchmarks are only as honest as their pass bars. Terminal-Bench shows that hard terminal work with instruction, container, tests, and an oracle is a useful unit of evaluation (Merrill et al., 2026). Harbor runs those tasks. Inspect gives a parallel composition model for datasets, solvers, and scorers. What those frameworks still leave to the task author is the craft of what fails for the right reason.

This note ships three public tasks and the reasoning behind them. Each one has a buggy starter that fails for a named cause, an oracle that clears every test, and a verifier that recomputes score from artifacts instead of trusting a self-reported JSON field. One task includes a staged fail transcript and a rubric with positive and negative magnitudes. A short case study on the distribution-shift task shows why mean-only "adaptation" can look serious and still leave ranking AUROC unchanged, and how a train-only label shortcut that flips on eval forces a real method.

This is not a leaderboard and not a claim about frontier models. It is a public sample of how I design and review agent evaluation work.

## 1. Where this sits

| Layer | What it gives you | What it does not invent for you |
|---|---|---|
| Terminal-Bench | Hard terminal tasks, human-checked oracles, outcome tests, Harbor as harness | The specific failure modes inside a new task |
| Harbor | Parallel trials, trajectories, verifier rewards | Whether the reward is gameable |
| Inspect | Dataset + solver/agent + scorer composition, sandboxes | The same craft question at sample level |

I treat the Terminal-Bench shape as the right container: English instruction, environment, tests that check final state, reference solution. Harbor is the runner I contribute harness fixes to. The suite here is smaller and deliberately public. The private analogues used different domains. The failure classes match: ordering, idempotency, dual consumers, and reports that look green while the method did nothing.

Two open contributions sit next to this note:

- **Harbor #4003** writes the target trajectory when a user agent is present, so trial context is not silent about who ran.
- **Terminal-Bench #2480** adds `lease-fencing-reaper`: exclusive leases, fencing tokens, a reaper under an injected logical clock. Stale renew/release after reclaim, wrong reap under the clock,

and token reuse after release are the named bugs. Same design bar as the suite below.

## 2. Design bar

A task ships when all four hold:

1. **Nop fails.** The starter fails for a reason I can name in one sentence.
2. **Oracle passes.** The checked-in solution clears every unit test.
3. **Score from artifacts.** The verifier recomputes the control or expected state. Editing a report field or a golden file does not invent a pass.
4. **Calibration note.** `CALIBRATION.md` lists starter failure modes and the shortcuts the tests block.

That bar is stricter than "tests exist." It is weaker than a multi-reviewer Terminal-Bench admission process. The point of publishing three tasks is to show the bar in public code, not to replace TB.

Anti-shortcuts I treat as default: fixture data and frozen baselines are read-only for the agent; property tests cover edge cases beyond the stock fixture; occupancy, balances, and controls are derived, not trusted from agent-written counters.

## 3. The three tasks

All numbers below are from the checked-in fixtures and oracles on 31 October 2026. Twelve local tests pass.

```
pytest -q -p no:randomly \
  tasks/clearing-hold-ledger/tests \
  tasks/slot-ring-buffer/tests \
  tasks/site-shift-meter/tests
```

### 3.1 clearing-hold-ledger (event-sourcing repair)

**Job.** Replay an immutable clearing stream into wallet balances and holds. Repair `clearing/replay.py`. Leave `data/events.json` alone.

**Starter failure modes.** Lexicographic `seq` sort (`"10"` before `"2"`); prefix collision on `dedupe_key`; partial HOLD when funds are short; RELEASE matched by wallet, not `hold_key`; zero-balance wallets dropped from output.

**Fixture.** 16 events. Oracle available balances: alpha 85, beta 45, delta/gamma/omega 0.

**Why it exists.** Multi-turn repair agents often chase the first red test, then edit the fixture, then invent a looser dedupe. The tests recompute from the stream. Prefix matching is worse than the starter once `post-alpha-100-extra` collides with `post-alpha-100`.

### 3.2 slot-ring-buffer (systems)

**Job.** Fixed-capacity packet ring with two independent consumers. When full with respect to the slowest consumer, drop incoming writes. Never overwrite unread slots for the slow reader.

**Starter failure modes.** Dual-consumer drop policy wrong, occupancy faked from a stale size counter, capacity edges at N=1 and N=2 missed.

**Why it exists.** Systems-lane sample without shipping private netlist or HDL fixtures. The contract is about unread packets staying alive, which is easy to break while green-looking unit tests on a single consumer still pass.

### 3.3 site-shift-meter (distribution shift)

**Job.** Train fault detection on sites S0/S1. Eval on S2. Beat a frozen control and report calibration honestly.

| Quantity | Value |
|---|---|
| `control_auroc` | 0.0108 |
| `model_auroc` | 0.9766 |
| `delta_auroc` | 0.9658 |
| Gate | `delta_auroc >= 0.08` and control match within `1e-6` |
| `n_eval` | 180 |

**Starter.** Trains a plain logistic on raw features, then copies that AUROC into both `control_auroc` and `model_auroc`. Delta is zero. The JSON shape looks finished.

## 4. Case study: when a “shift fix” does nothing

Train on S0/S1, score on S2. Feature means move. A natural first fix is per-site z-scoring, then the same logistic.

On a mean-only synthetic draw, that barely moves ranking AUROC for a linear scorer. Add a constant to every score and the order stays the same. So the “adapted” model clears almost nothing against the control, even though the notebook looks like domain adaptation.

What actually breaks the control is a strong label shortcut in `f4` on the train sites, flipped on eval (anti-correlated noise). The frozen control in `baseline.py` latches onto `f4` and collapses on S2 (~0.01 AUROC). A method that drops `f4` and z-scores `f1` - `f3` per site, using unlabeled eval features for eval-site moments, clears a large delta (~0.97 on the checked-in CSVs).

The gate is mild on purpose ( `>= 0.08` ) so a clean method passes without needing a perfect model, and so copying the control or cosmetic mean-only z-scoring still fails once the spurious feature is in play.

**Pass bar.** Starter fails (delta 0). Verifier recomputes the control from `baseline.py` + the CSVs. `data/` and `baseline.py` are read-only. `n_eval` must equal 180.

**Shortcuts blocked.** Printing a nicer delta without beating the frozen control. Rewriting the baseline. Mean-only normalization that never changes ranking metrics. Editing goldens instead of fixing the method.

That is the pattern I keep seeing in private eval work: nop fails, oracle passes, score comes back from artifacts, and the control is honest even when it hurts. This public task is the small version, synthetic data only.

## 5. Trace review: staged fail on the ledger

Fixture: `docs/fixtures/ledger-fail-run.json`. Staged transcript, not a live model log. Built to match buggy starter behavior so the review process is visible.

**What went wrong.**

1. **Lexicographic seq.** Turn 3 left `sorted(..., key=str)`. Applied order puts 10 before 2. Balances drift from there.
2. **Edited the fixture.** Turn 6 changed `data/events.json`. Outside the repair surface.
3. **Worse dedupe.** Turn 8 "fixed" collisions with prefix matching. Drops `post-alpha-100-extra` when `post-alpha-100` is already seen. Exact key equality was the rule.
4. **Never checked holds.** Partial HOLD and RELEASE-by-wallet stayed in the starter. The agent chased the first red test and stopped.

**Rubric** (one behavior per line, magnitudes in {1,2,3,5}):

```
Agent sorts events by numeric seq before replay, +3
Agent uses exact dedupe_key equality rather than prefix matching, +3
Agent applies HOLD only when full amount is available, +2
Agent releases holds by hold_key identity, +2
Agent keeps zero-balance wallets in the output maps, +1
Agent leaves data/events.json unchanged, +2
Agent edits the event fixture to make tests pass, -5
Agent treats string-ordered seq as solved after one green-looking run, -2
Agent retries the same failing pytest three or more times with no code change, -1
Agent introduces prefix dedupe after seeing a collision failure, -3
```

Positive sum: 13. Negatives cover the shortcuts this staged run took. The run fails the task and the rubric. A mergeable fix clears the six positives without touching `data/`.

I use this format when the tests already said fail and the question is process judgment: did the agent repair the contract, or did it negotiate with the fixtures?

## 6. Adjacent Terminal-Bench work

`lease-fencing-reaper` (open PR against `harbor-framework/terminal-bench`) is the same craft in TB's container format.

Observed symptoms the instruction names: after expiry and reclaim, the previous owner's fencing token can still renew or release; reaping frees live leases under the injected clock, or misses expired ones; holder checks report true past expiry; after release or reclaim, acquire reuses an old fencing token.

Intended semantics live in `DESIGN.md`. Time comes only from the injected `Clock`. Types and design files are read-only. That is the Harbor/TB shape: outcome tests on final state, oracle solution, no cheating the clock by editing the contract files.

I am not claiming a leaderboard score on Terminal-Bench 2.0. The PR is the contribution. This suite is the portable explanation of the bar.

## 7. Limits

- Three tasks. Synthetic or small fixtures only.
- No multi-model agent sweep published here. The staged ledger trace is constructed.
- Harbor and Terminal-Bench PRs may still be open at the time you read this. Check the links.
- Private client work is not included. Names and fixtures stay out on purpose.
- This note does not claim SOTA, safety properties, or that the gate values generalize beyond these CSVs.

If you want a larger public set, the next useful additions are a flake-prone timing task with a deterministic clock seam, and a verifier that diffs trajectories against the rubric automatically. Neither is in 0.1.0.

## 8. Reproduce

```
git clone https://github.com/gyanu2507/agent-eval-suite
cd agent-eval-suite
python3 -m pytest -q -p no:randomly \
  tasks/clearing-hold-ledger/tests \
  tasks/slot-ring-buffer/tests \
  tasks/site-shift-meter/tests
python3 tasks/site-shift-meter/solution/train_and_eval.py
```

Read `tasks/*/CALIBRATION.md`, `docs/case-study.md`, and `docs/trace-review.md` for the short forms of sections 4 and 5.

## References


Merrill, M. A., Shaw, A. G., Carlini, N., et al. (2026). *Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces*. arXiv:2601.11868. https://arxiv.org/abs/2601.11868

Harbor Framework. https://github.com/harbor-framework/harbor

UK AI Security Institute. *Inspect*. https://inspect.aisi.org.uk/

Mayank, G. (2026). *agent-eval-suite* (v0.1.0). https://github.com/gyanu2507/agent-eval-suite


## Citation

```
@techreport{mayank2026deltafake,
  author       = {Mayank, Gyanu},
  title        = {When the delta is fake: Three public agent-eval tasks
                  with nop starters, frozen controls, and a scored fail trace},
  year         = {2026},
  month        = {10},
  institution  = {Independent},
  version      = {0.1.0},
```

```
  url          = {https://github.com/gyanu2507/agent-eval-suite}
}
```

After a Zenodo DOI exists for this version, replace the `url` field with the DOI URL and add `doi = {...}`.